Image Classification on CIFAR-10 with Random Forests

MarcAnthony Cantu

March 2026

Load CIFAR dataset

The CIFAR dataset was loaded from the OpenML database. The datum in this set contains 32x32 images of certain object categories such as cats, airplanes, etc. The dataset was loaded into 2 different arrays

$X \in \mathbb{R}^{50000 \times 3072}$ contains the feature values for all 50000 cases
$Y \in \mathbb{R}^{50000}$ contains the labels or class identifiers as integers from 0-9 where y_n is the true label for x_n

Select 3 classes and display cases

Among the 10 classes in Y 3 were selected and the remaining were dropped along with their corresponding feature vectors in X. The 3 cases I chose were $Y = [3, 6, 9]$ or more explicitly, $Y = [\text{cat} \ , \text{frog} \ , \text{truck}]$.
3 cases were chosen from each and displayed:

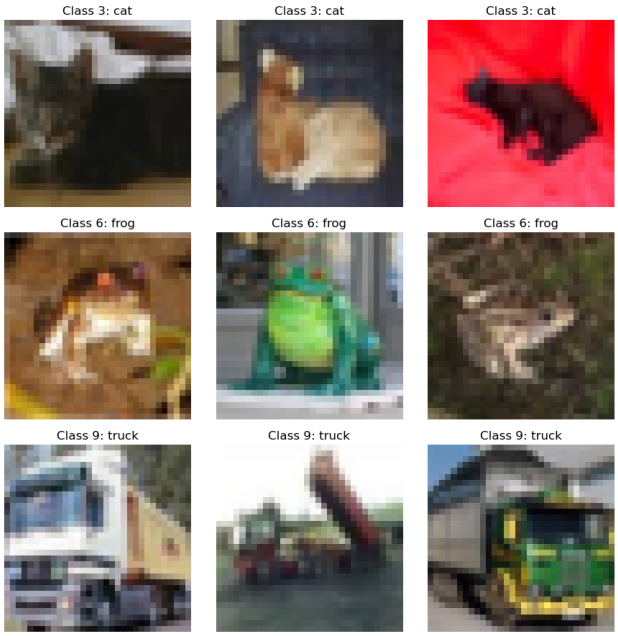


Figure 1: 3 Cases from each class

Main Algorithms Involved in a Random Forest Classifier

A Random Forest classifier is an ensemble learning method that constructs a large collection of decision trees and combines their predictions to improve classification accuracy and reduce overfitting. The method

relies on several key algorithms: decision tree learning, bootstrap aggregation (bagging), and random feature selection.

1. Decision Tree Learning

Each tree in the forest is constructed using a decision tree algorithm. A decision tree recursively partitions the feature space into regions that contain mostly samples of a single class.

At each node of the tree:

- A subset of training samples reaches the node.
- The algorithm evaluates possible splits of the data based on the feature values.
- A split is chosen that maximizes the reduction in impurity.

For classification tasks, impurity is commonly measured using the **Gini impurity**.

Gini Impurity

$$G = 1 - \sum_{k=1}^{K} p_k^2$$

where p_k is the proportion of samples belonging to class k in the node.

The tree grows by repeatedly selecting the split that produces the largest reduction in impurity. Splitting continues until a stopping condition is met, such as reaching a maximum depth or having too few samples in a node.

2. Bootstrap Aggregation (Bagging)

Random Forest uses a technique called **bootstrap aggregation**, or **bagging**, to create multiple diverse training datasets.

The algorithm proceeds as follows:

1. Given a dataset of n samples, a bootstrap dataset of size n is generated by sampling from the original data with replacement.
2. A decision tree is trained using this bootstrap dataset.
3. This process is repeated many times to create a collection of trees.

Because each tree is trained on a slightly different dataset, the trees become diverse. Averaging their predictions reduces the variance of the model and improves generalization performance.

3. Random Feature Selection

Another important source of randomness in Random Forest is feature selection during tree construction.

At each split in a tree:

- Instead of considering all p features, the algorithm randomly selects a subset of m features.
- The best split is chosen only from this subset.

For classification problems, a common choice is

$$m = \sqrt{p}$$

where p is the total number of features.

This randomness prevents trees from becoming highly correlated and increases the diversity of the ensemble.

4. Ensemble Prediction

After all trees are trained, the Random Forest produces predictions by combining the outputs of the individual trees.

For classification tasks, this is typically done using **majority voting**:

$$\hat{y} = \text{mode}(T_1(x), T_2(x), \ldots, T_B(x))$$

where $T_i(x)$ is the prediction of the i-th decision tree and B is the number of trees in the forest.
The class that receives the most votes across all trees becomes the final prediction.

scikit_learn Random Forest function

The RamdomForestClassifier has many parameters:

n_estimator

- input: int, default=100
- The number of trees in the forest

criterion

- input: {"gini", "entropy", "log_loss"}, default="gini"
- Function that measures impurity of the split

max_depth

- input: int, default=None
- The maximum depth of the tree. If None, then nodes are expanded until all leaves are pure or until all leaves contain less than min_samples_split samples

min_samples_split

- input: int or float, default=2
- The minimum number of samples required to split an internal node
  - If input is an integer z then minimum samples is z
  - If input is a float q then minimum samples is $\lceil q \times n \rceil$ (value rounded up) where n is the total number of samples

min_samples_leaf

- input: int or float, default=1
- The minimum number of samples required to be at a leaf node
  - If input is an integer z then minimum is z
  - If input is a float q then minimum samples is $\lceil q \times n \rceil$ where n is the total number of samples

min_weight_fraction_leaf

- input: float, default=0.0
- The minimum weighted fraction of the sum total of weights (of all the input samples) required to be at a leaf node. Samples have equal weight when sample_weight is not provided

max_features

- input: {“sqrt”, “log2”, None}, int or float, default=”sqrt”
- Number of features to consider when splitting
  - If input is ”sqrt” then number of features is $\sqrt{m}$ where m is total number of features
  - If input is ”log2” then number of features is $log_2(m)$ where m is total number of features
  - If input is an integer z then number of features is z where m is total number of features
  - If input is a float q then number of features is $max\{1, int(q \times m)\}$ where m is total number of features

max_leaf_nodes

- input: int, default=None
- Grow trees with max_leaf_nodes in best-first fashion. If None then unlimited number of leaf nodes

min_impurity_decrease

- input: float, default=0.0
- A node will be split if this split induces a decrease of the impurity greater than or equal to this value

bootstrap

- input: bool, defualt=True
- Whether bootstrap samples are used when building trees. If False, the whole dataset is used to build each tree

oob_score

- input: bool or callable, default=False
- Whether to use out-of-bag samples to estimate the generalization score. By default, `accuracy_score` is used. Provide a callable with signature `metric(y_true, y_pred)` to use a custom metric.

n_jobs

- input int, default=None
- The number of jobs to run in parallel

random_state

- input: int, `RandomState` instance or None, default=None
- Controls randomness of bootstrapping and sampling of features. Setting the input to an integer allows for reproducibility of the forest

verbose

- input: int, default=0
- Controls verbosity when fitting and predicting

warm_start

- input bool, default=False
- When True, reuse the solution of previous call to fit and add more estimators to the ensemble

ccp_alpha

- input non-negative float, default=0.0
- Complexity parameter used for Minimal Cost-Complexity Pruning. The subtree with the largest cost complexity that is smaller than `ccp_alpha` will be chosen

max_samples

- input: int of float, default=None
- If bootstrap is True, gives number of samples to draw from the training set

Split classes into training set and testing set and developing random forest

Both the data set X and the label set Y were split into respective training and testing sets:

$$X_{train} \cup X_{test} = X$$
$$Y_{train} \cup Y_{test} = Y$$

They were split in such a way so that for each class C_i 85% of C_i is contained in the training set and 15% is contained in the testing set.
We then create a random forest RF_{100} to fit our training data on. The parameter values for our forest are:

```
  params = {
    "n_estimators": 100
    "criterion": "gini",
    "max_depth": None,
    "min_samples_split": 2,
    "min_samples_leaf": 1,
    "max_samples": None,
    "max_features": "sqrt",
    "n_jobs": = -1
    "max_leaf_nodes": None,
}
```

After fitting the training set, the test and training accuracies were computed:

$$acc_{train} = 1$$
$$acc_{test} = 74.27\%$$

Having the parameter `n_jobs` $= -1$ allows us to cut the computation time significantly from **45 seconds to 5 seconds**

Confusion Matrix

The confusion matrix of $RF1$ was created as seen below:

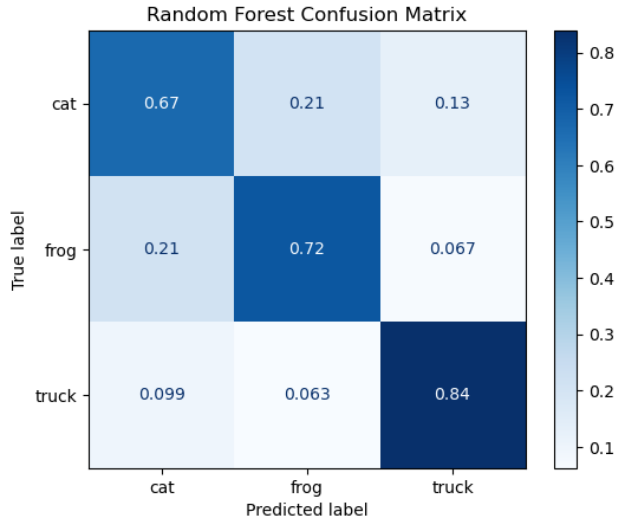


Figure 2: Confusion Matrix for $RF1$

As we can see, our random forest has done a fairly decent job at predicting our test set. It was most accurate at predicting the class "trucks" and most inaccurate are predicting the class "cats" with the majority of its misclassifications being "frogs".
This makes sense since the class trucks will likely appear vastly different in comparison to cats and frogs. However, cats and frogs appear similar depending on the angle or colors. To investigate further, let's look at some misclassified cases:



Figure 3: 3 cases that were misclassified

It is quite interesting to view these images and infer why the misclassifications occurred. In particular, the last case appears to be a cat perched on some kind of furniture which is likely the culprit in causing the misclassification as a truck.

Hyper-parameterization of our Random Forest

Among the parameters listed above, different values were testing of each to find an optimal forest model. The values tested for each parameter are:

```
    options = {
    "criterion": ["gini", "entropy", "log_loss"],
    "max_depth": [None, 10, 20, 30, 50],
    "min_samples_split": [2, 5, 10, 20],
    "min_samples_leaf": [1, 2, 4],
    "max_features": ["sqrt", "log2", 20, 50, 100, 150],
    "max_leaf_nodes": [None, 10, 20, 30],
    "max_samples": [None, 0.25, 0.5, 0.75]
}
```

After many iterations of parameter settings we were able to increase the accuracy by only a marginal percentage with our test accuracy going from **74.21%** to **75.16%**.
The parameters that were updated were:

Parameter	old value	new value
min_sample_split	2	10
max_features	"sqrt"	100
Accuracies	74.21%	75.16%

Table 1: Updated Parameters

Certain parameters that greatly affected the accuracy of the forest were:

- `max_leaf_nodes` which depreciated the accuracy as the inputted value got smaller. The largest accuracy loss was at `max_leaf_nodes` = 10 with an accuracy difference of 10.22%. Evidently, putting a cap on the allotted leaf nodes only hurts the accuracy of the tree.
- `max_samples` similarly lowered accuracy as the samples size got smaller with the largest accuracy difference being 2.31%

Create RF_{oob} using whole dataset

A new random forest was created with the updated parameters and then fitted to the entire dataset X rather than the training set X_{train}. We then measured its accuracy using the out-of-bag estimate by setting the parameter `oob_score`=True and compared to the accuracy of our RF_{100} model.

$$acc_{100} = 75.16\%$$
$$acc_{oob} = 73.04\%$$

The RF_{oob}'s accuracy is lower by a difference of 2.12%

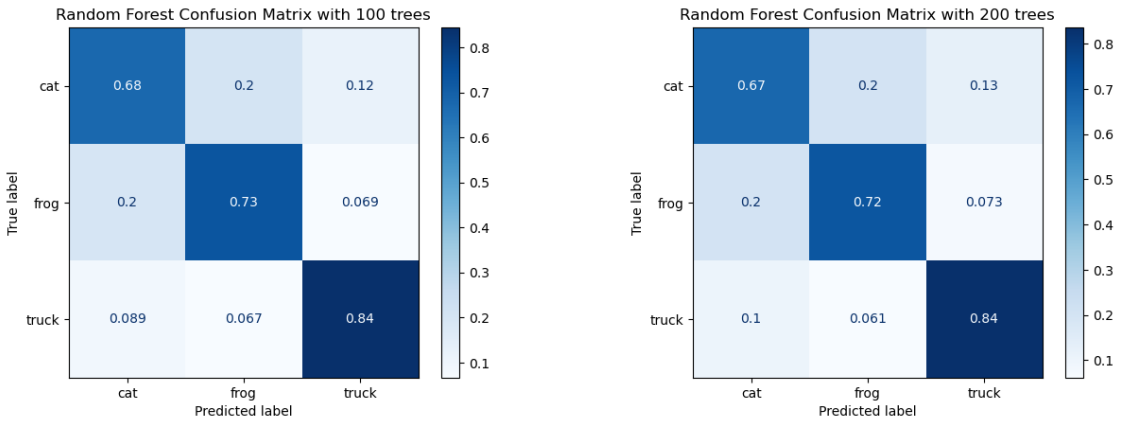
Creating Random Forests with bag sizes of {200, 400, 600}

Random Forests with varying number of trees were created and fitted to the training set to compare accuracies. The values tested were {100, 200, 400, 600} and yeilded the following accuracies:

number of trees	test accuracy
100	75.16%
200	74.40%
400	75.16%
600	74.71%

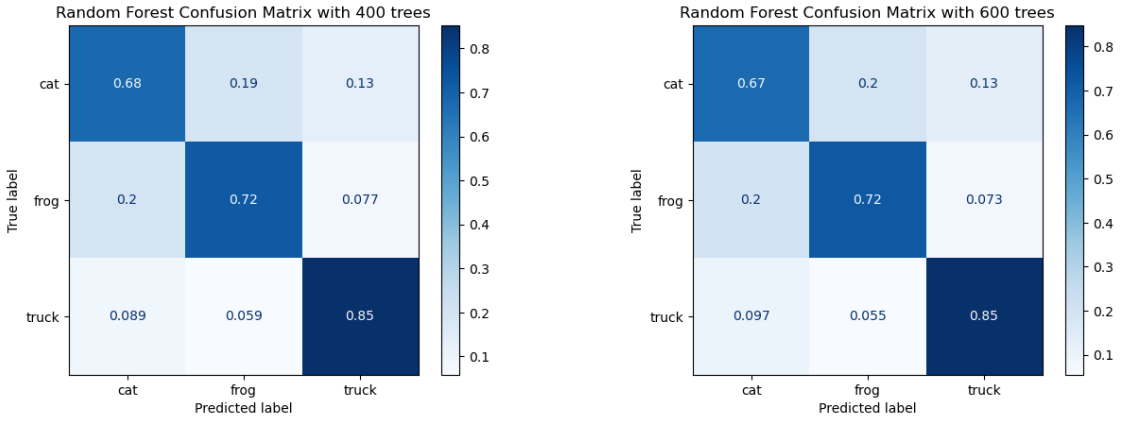
Table 2: Accuracies of each forest

As we can see, none out preformed RF_{100}. Interestingly, however, RF_{400} preformed equally to RF_{100} with an equivalent accuracy. Looking at the confusion matrices we can see how the accuracies are distributed for each forest among the 3 classes:



(a) Confusion Matrix for RF_{100}

(b) Confusion Matrix for RF_{200}



(c) Confusion Matrix for RF_{400}

(d) Confusion Matrix for RF_{600}

Figure 4: Confusion Matrices for all 4 forests

Evaluating importance of the color channel features

Since our highest accuracy still remains to RF_{100} we stick with this as our primary model. We now will evaluate the importance of each color channel (Red, Green, Blue) as a feature in our random forest. We start by creating 3 alternative datasets each one with a different color channel stripped from it X_{GB} has no red, X_{RG} has no green, and X_{RG} has no blue. Creating training and testing sets for each of these we then fit

our forest RF_{100} to each training set and measure the accuracies denoted acc_p where p is the missing color channel. The importance of any given feature p is computed by:

$$importance(p) = acc_{100} - acc_p$$

Here are the results:

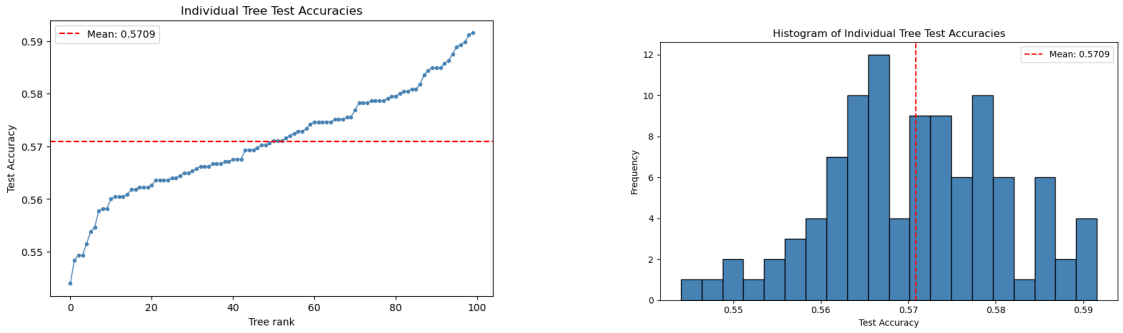
Missing channel	Accuracy	Importance
Red	70.40%	4.76%
Green	70.71%	4.44%
Blue	70.93%	4.22%

Table 3: Importance of each Color Channel

From these results we can see that the importance of the colors are nearly evenly distributed. This implies that the color distributions among each class are likely very similar and thus play little part in the decision process of our forest relative to one another.

Computing accuracies of each tree in RF_{100}

We will now evaluate the accuracies of each individual tree in our random forest. Each of the 100 trees were tasked to predict the test set X_{test} and the accuracies were recorded and stored into an array. The array was than sorted from lowest accuracy to highest accuracy and graphed via plot and histogram:



(a) Plot of tree Accuracies

(b) Histogram of tree accuracies

Figure 5: Accuracies of individual trees

Pruning the Random Forest RF_{100}

Using the above results we took removed the 20 lowest ranking trees from our forest and created a new pruned random forest RF^*_{80} which contained only the top 80 best preforming trees from RF_{100}. Using RF^*_{80} to predict X_test yielded slightly lower accuracy:

$$acc_{80} = 74.98\%$$
$$acc_{100} = 75.16\%$$
$$acc_{100} - acc_{80} = 0.18\%$$